Vendor-Neutral AFK Control Plane for Cross-Vendor AI Coding Agents

Enhanced Project Definition, Architecture, Services, Security, and Implementation Blueprint

Document status: Comprehensive project specification

Revision: 2 — Architecture Hardening Pass

Prepared from: Enhanced proposal supplied by the project owner

Primary objective: Define a secure platform that allows developers to launch, supervise, guide, approve, and review locally executed AI coding-agent tasks from a web browser or mobile device, regardless of the agent vendor.

---

1. Executive Overview

The project is a **vendor-neutral remote control and orchestration platform for local AI coding agents**. It addresses a growing limitation in modern software-development workflows: coding agents can perform long-running work on a developer’ s computer, but remote-control capabilities are usually tied to a particular vendor, editor, or cloud environment.

The proposed platform separates the system into two major parts. The first is a **Local Agent Gateway**, installed on the developer’ s workstation, which launches and supervises coding agents such as Codex CLI, Claude Code, OpenCode, Cline, Cursor, and Antigravity. The second is a cloud-hosted **Control Plane**, accessed through a web or mobile interface, which authenticates users, pairs devices, relays encrypted messages, applies approval policies, sends notifications, and records an auditable history.

The essential design principle is local-first execution. The agent, source code, shell commands, filesystem operations, tests, and repository access remain on the user’ s machine. The cloud is a coordination and presentation layer. Data leaving the machine is encrypted, passed through a secret-redaction layer, and limited to the information necessary for remote supervision. In addition, every agent process must run inside a session-specific sandbox before it can access project files, execute commands, or communicate with external services.

> **Core value proposition:** Start a task with one AI coding agent or several agents on a local workstation, supervise the work from anywhere, review its output on a phone, approve

> sensitive operations, and preserve a complete tamper-evident history without surrendering control of the local development environment.

The enhanced revision materially strengthens the original proposal. Sandboxing, device revocation, tamper-evident audit logs, mandatory approval floors, redaction, service separation, rate limiting, tenant isolation, and reconnect reconciliation are no longer optional future improvements. They are part of the architecture and must be present before the product is considered ready for general use.

---

2. Project Vision and Goals

2.1 Vision

The long-term vision is to become the common **remote supervision and safety layer for local AI software-development agents**. Users should be able to choose their preferred coding agent while receiving a consistent experience for device management, task control, terminal streaming, diff review, approvals, notifications, integrations, and auditing.

The product should not attempt to replace every coding agent. Instead, it should standardize the layer underneath them. Agent vendors remain responsible for their own models and agent behavior; this platform provides a common execution boundary, control protocol, policy layer, and user interface.

2.2 Primary goals

Goal	Description
Cross-vendor control	Support multiple local coding agents through independent adapters and expose them through one interface.
Local execution	Keep code, files, shells, tests, and agent processes on the user’ s machine.
Secure remote access	Allow web and mobile clients to control local sessions without opening inbound ports on the workstation.
Safe autonomy	Permit low-risk automation while enforcing human approval for sensitive actions.
Mandatory isolation	Run every agent session inside a scoped sandbox with least privilege and default-deny network egress.

Transparent review	Stream progress, display diffs, and support commit or pull-request review from a mobile device.
Accountability	Maintain a timestamped, append-only, hash-chained history of actions and decisions.
Extensibility	Add new agents by implementing adapters rather than modifying the gateway core.
Team readiness	Support organizations, roles, policy enforcement, administrative escalation, and SIEM export.

2.3 Non-goals for the first release

The first release should not attempt to build a new foundation model, replace a full desktop IDE, provide unrestricted remote shell access, or guarantee support for every editor and agent immediately. It should also avoid treating cloud execution as equivalent to local execution. The product’ s differentiation depends on controlling locally running agents while adding a secure remote interface.

---

3. Problem Statement

AI coding agents can plan changes, edit files, run tests, refactor code, inspect repositories, and perform operational tasks. These capabilities make them useful for long-running work such as large refactors, overnight test suites, dependency maintenance, bug repair, and migration preparation. However, an agent may require supervision at any time, especially when it reaches a destructive command, encounters an ambiguous requirement, or proposes changes that need review.

Current remote-control experiences are fragmented. Some vendors provide a remote interface only for their own agent, while other agents expose a CLI or local API without a mobile or web interface. A developer who uses multiple agents must operate multiple authentication methods, dashboards, policies, and approval systems. This creates operational friction and makes it difficult to enforce one security model across tools.

The project addresses four related problems:

1. **Remote supervision:** Developers cannot reliably start or continue local agent sessions while away from the workstation.
2. **Vendor fragmentation:** Each agent exposes different control methods, event formats, and approval behavior.

3. **Safety risk:** An agent can execute shell commands, modify files, access credentials, or communicate with external infrastructure.
4. **Insufficient accountability:** Existing local workflows may not provide a centralized, immutable record of commands, approvals, diffs, state transitions, and device actions.

The platform must therefore provide remote access without turning the cloud into an uncontrolled execution environment or weakening the local machine’ s security boundary.

---

4. Users and Representative Scenarios

4.1 Target users

User group	Primary need
Remote and hybrid developers	Start and supervise local work from a phone, tablet, or secondary computer.
Individual developers	Run overnight maintenance tasks and receive alerts only when action is required.
Managers and technical leads	Review team sessions, approve production-affecting actions, and monitor progress.
Security and DevOps teams	Enforce organization-wide policies, restrict risky operations, and export audit events.
Platform engineering teams	Integrate agent tasks into CI/CD, repositories, and internal tooling.

4.2 Core user journey

A user installs the Gateway on a workstation, authenticates to the Control Plane, and pairs the workstation through a short-lived code or QR flow. The Gateway creates its device identity locally and displays a fingerprint. The user verifies that the fingerprint shown locally matches the fingerprint displayed by the trusted application. Only then is the device marked trusted.

The user selects a paired workstation and project, chooses an agent, and enters a task. The Control Plane creates a session record and sends the request through the encrypted broker connection. The Gateway starts the selected adapter, creates a sandbox, launches the agent inside it, and begins streaming redacted output. If the agent requests a protected action,

the session pauses and the user receives an approval request. When the task completes, the user can inspect the diff, commit changes, or open a pull request.

4.3 Representative scenarios

Remote test repair. A developer starts “repair all failing tests” from a phone. The agent runs locally, streams progress, modifies only the project directory, and pauses before committing. The developer reviews the diff and approves the commit.

Overnight maintenance. A developer starts dependency analysis before going to sleep. The task runs with a restricted trust profile. The system sends a notification only when the task completes, fails, or needs approval.

Production protection. An agent prepares an infrastructure change. It may inspect files and run a staging dry run, but a production-tagged infrastructure apply remains subject to the deny-override approval floor even when the session uses an autonomous profile.

Team review. A team lead opens a colleague’s session, reviews the planned changes, and approves a permitted deployment action according to the organization’s role and policy model. The access and approval are recorded in the audit trail.

---

5. Product Scope and Capability Model

The platform exposes a common set of capabilities across different agents. Not every agent will support every capability equally, so the adapter must declare its capabilities explicitly.

Capability	Description
Session creation	Start a new agent session in a selected project and sandbox.
Prompt delivery	Send user instructions to the active agent.
Interactive input	Send follow-up messages, confirmations, or corrections.
Output streaming	Stream agent text, terminal output, progress events, and summaries.
Diff collection	Capture changed-file diffs for review.
Approval interception	Detect or mediate protected actions before execution.
Cancellation	Stop the agent process and clean up its session resources.

Resume and reconnect	Reconcile local and cloud state after a network interruption.
Multi-session support	Run several agent sessions on one or more devices.
Repository integration	Commit, push, or open a pull request subject to policy.
Capability declaration	Tell the Control Plane which features an adapter supports.

The adapter contract should be versioned. A suggested uniform interface is:

Plain Text

```
installOrDetect()
validateEnvironment()
startSession(sessionConfig)
sendMessage(sessionId, message)
streamEvents(sessionId)
requestApproval(sessionId, action)
abortSession(sessionId, reason)
collectDiff(sessionId)
getStatus(sessionId)
cleanupSession(sessionId)
```

The Gateway core should never contain vendor-specific assumptions. Agent-specific behavior belongs inside an adapter package that conforms to the versioned contract.

---

6. High-Level Architecture

6.1 Architectural structure

The system consists of three operational zones:

1. **User clients:** Web browser and native or progressive mobile application used to view sessions and issue commands.
2. **Cloud Control Plane:** Authentication, device management, message brokering, policy evaluation, notifications, durable state, audit, and integration services.
3. **Developer workstation:** Gateway, adapters, sandbox runtime, redaction proxy, local checkpoint store, and agent processes.

The workstation makes outbound connections only. The Gateway does not open an inbound Internet port. This significantly reduces exposure to unsolicited network traffic and allows the platform to function behind common routers and firewalls.

6.2 Logical data flow

Plain Text

```
Web/Mobile Client
       |
       | HTTPS / WebSocket
       v
API and WebSocket Gateway
       |
       +--> Auth Service
       +--> Policy and Permission Engine
       +--> Message Broker
       +--> Audit Service
       +--> Integration Services
       |
       | End-to-end encrypted message envelope
       v
Local Agent Gateway
       |
       +--> Adapter Manager
       +--> Per-session Sandbox
       +--> Secret Redaction Proxy
       +--> Local Checkpoint Store
       |
       v
Sandboxed Agent Process
       |
       +--> Project workspace
       +--> Restricted tools
       +--> Controlled network egress
```

Every crossing of the workstation boundary must use authenticated encryption. The Control Plane may route ciphertext and store metadata needed for delivery and indexing, but it must not require plaintext source code or secrets to perform its coordination role.

---

7. Local Agent Gateway

The Agent Gateway is the trusted local runtime installed on a developer’ s machine. It is responsible for connecting local agents to the Control Plane while enforcing the machine-

side security boundary.

7.1 Gateway responsibilities

The Gateway discovers installed agents, loads signed adapters, validates project configuration, creates sandboxed sessions, relays commands, captures output, performs redaction, maintains the encrypted tunnel, persists recovery checkpoints, and reports health metrics.

It also acts as the final local enforcement point. The client interface must never be allowed to bypass the Gateway, and an agent must never be allowed to execute outside the sandbox configured for its session.

7.2 Gateway components

Component	Responsibility
Gateway Core	Lifecycle management, command routing, session registry, configuration, and error handling.
Adapter Manager	Loads, verifies, starts, and stops agent adapters.
Agent Adapters	Translate the common platform protocol into an agent’ s CLI, HTTP, SDK, or process behavior.
Sandbox Manager	Creates and destroys per-session isolation environments.
Policy Enforcement Client	Applies cloud policy decisions locally before protected operations proceed.
Secret Redaction Proxy	Removes credentials and sensitive paths from outbound text, diffs, and events.
Tunnel Client	Maintains mutually authenticated, encrypted communication with the Control Plane.
Checkpoint Store	Persists message sequence numbers, local event records, and recovery state.
Health and Metrics Module	Reports session counts, resource usage, latency, and connection health.

Secure Updater	Installs only signed and integrity-verified Gateway and adapter releases.

7.3 Adapter design

Each adapter handles the differences between agents. For an HTTP-native agent, the adapter may call a local API and subscribe to an event stream. For a CLI-only agent, it may spawn a child process, write prompts to standard input, read standard output and standard error, and translate process events into the platform’ s normalized event model.

Adapters must be isolated from one another. A compromised or malfunctioning adapter should not automatically gain access to another adapter’ s process, credentials, session files, or project scope. Adapter packages should be signed, version-pinned, and validated before installation.

7.4 Local API

The Gateway may expose a localhost API for local integrations, IDE extensions, and diagnostic tools. This API must bind only to loopback by default, use authentication even on localhost, validate all input, enforce rate limits, and apply the same policy checks as the cloud path. Localhost must not be treated as inherently trustworthy because other processes on the workstation may be malicious or compromised.

---

8. Mandatory Sandbox and Isolation Layer

Sandboxing is a launch requirement rather than a later enhancement. Every agent session must begin inside an isolation boundary before the agent can access project files, run shell commands, or access a network.

8.1 Sandbox properties

A session sandbox should provide:

- A filesystem view limited to the declared project directory and explicitly approved temporary paths.
- No root or administrator privileges.
- A distinct process namespace or equivalent isolation mechanism where supported.
- Default-deny outbound network access.
- Explicit allowlists for required package registries, source-control systems, model APIs, or deployment endpoints.

- Resource limits for CPU, memory, process count, disk usage, and execution time.
- Separate credentials scoped to the session and injected only when needed.
- Cleanup of temporary files and processes when the session ends.

The implementation may use containers on platforms where a reliable container runtime is available. Where containers are not practical, the Gateway should use operating-system sandboxing, restricted user accounts, filesystem permissions, process isolation, and a network firewall. The product must clearly document platform-specific security differences rather than claiming identical isolation on every operating system.

8.2 Project scope

A session should include an explicit project root, repository identity, branch, and writable-path policy. Attempts to read or write outside the permitted scope must be blocked or routed to approval. The project root should not be inferred solely from an arbitrary user-provided string; it should be selected from Gateway-discovered workspaces or confirmed through the local interface.

8.3 Network egress

The default network policy is deny. A task can request a network capability such as access to a package registry or a Git provider. The Policy Engine evaluates the request, and the Gateway enforces the resulting allowlist. Network access should be represented as an auditable capability grant with a start time, expiry, destination, and reason.

---

9. Secret Redaction and Data-Handling Model

The Secret Redaction Proxy is placed directly in the outbound data path. Agent output, terminal logs, diffs, error messages, and file events must pass through this component before reaching the tunnel.

Redaction should combine pattern detection with user and organization configuration. Recognizable API keys, bearer tokens, private-key material, passwords, connection strings, cloud credentials, and values from configured secret paths should be replaced with stable placeholders such as `[REDACTED_SECRET]`.

Redaction is a defense layer, not a guarantee that an arbitrary secret can never be encoded in an unusual form. The system should therefore minimize the data it transmits, block configured sensitive paths, avoid sending raw environment dumps, and provide administrators with configurable data-loss-prevention rules.

Data class	Default treatment

Agent status and lifecycle events	Send as encrypted metadata.
Normalized text output	Redact, encrypt, and stream where required.
Source-code diff	Redact, encrypt, and store according to retention policy.
Credential values	Never transmit; replace locally with a placeholder.
Environment-variable listing	Block by default; require explicit policy and approval if supported.
Files outside project scope	Block or request approval; do not stream by default.
Audit metadata	Store as structured, hash-chained events.

The project should offer a configurable retention policy. Individual users may need short retention, while enterprise users may need longer audit retention and external SIEM export. Retention must not silently convert encrypted transient data into permanent plaintext storage.

---

10. Cloud Control Plane Services

The Control Plane is a set of logical services. They may initially be deployed as a modular monolith for speed, but their responsibilities and security boundaries should be defined as if they were independent services. This allows future extraction without redesigning the domain model.

10.1 API and WebSocket Gateway

This is the externally reachable application boundary. It accepts authenticated client requests, validates schemas, applies request limits, authorizes access to workspaces and sessions, and establishes WebSocket connections for live updates.

It should not directly make policy decisions by duplicating business rules. Instead, it calls the Policy Engine and records important actions through the Audit Service. It should also not mint device certificates; that responsibility belongs to the key-management boundary.

10.2 Authentication Service

The Auth Service manages user login through OIDC and supports mandatory multi-factor authentication. It issues user sessions and access tokens for the web and mobile clients.

The service should support account recovery, session revocation, suspicious-login detection, and organization membership checks.

Future enterprise functionality may include SSO, SCIM provisioning, hardware-backed authentication, and FIDO2 approval confirmation.

10.3 Device and Key Management Service

The Key Management Service manages device identities, certificate issuance, certificate rotation, revocation, and pairing state. The Gateway generates its private key locally; the private key must never be sent to the Control Plane.

The KMS should support short-lived device certificates, explicit device revocation, certificate status checking, and key rotation. A revoked device must lose its ability to establish or maintain the tunnel, and the Control Plane must terminate active sessions or mark them unavailable according to policy.

10.4 Message Broker

The broker routes commands, events, approval requests, and acknowledgements between clients, Control Plane services, and Gateways. At-least-once delivery is preferred over silent loss, but it requires idempotency keys, sequence numbers, acknowledgement records, and duplicate-event handling.

The broker should distinguish between ephemeral presence data and durable task events. A transient typing indicator can be lost; an approval decision or session-completion event cannot be silently lost.

10.5 Policy and Permission Engine

The Policy Engine is the single source of truth for protected-action decisions. It evaluates:

- User identity and role.
- Workspace and organization policy.
- Project and repository policy.
- Agent identity and adapter capabilities.
- Session trust profile.
- Requested action category.
- Target path, branch, environment, or network destination.
- Immutable deny-override rules.
- Approval timeout and escalation settings.

A decision should be explainable and auditable. The engine should return a structured result containing the decision, reason, matched rules, required approver role, expiry, and a policy version.

10.6 Audit Service

The Audit Service receives all security-relevant events and appends them to a hash chain. Each event contains the event payload, timestamp, actor, device, session, sequence number, previous-event hash, and current-event hash.

The service should expose chain verification, export, retention management, and integration with external SIEM systems. Audit records must be append-only from the application perspective. Administrative corrections should create a new compensating event rather than modifying the original record.

10.7 Notification Service

The Notification Service converts important events into push notifications, email messages, Slack messages, or Teams messages according to user and workspace preferences. It must avoid sending secrets or sensitive source code in notification payloads. Notification content should contain a short summary and a deep link to the authenticated application.

10.8 Integration Service

The Integration Service manages OAuth connections and provider-specific actions for GitHub, GitLab, Bitbucket, CI/CD systems, Slack, Teams, and future cloud providers. Tokens should be stored using encrypted secret storage and used through narrowly scoped provider permissions.

10.9 Durable and ephemeral storage

A relational database such as PostgreSQL should store users, organizations, workspaces, devices, projects, sessions, policies, approvals, diffs, integrations, and audit indexes. Redis or an equivalent system should store presence, short-lived approval locks, rate-limit counters, and broker coordination data.

---

11. Device Pairing and Trust Establishment

Pairing is the process that binds a physical workstation to an authenticated user or workspace.

11.1 Pairing flow

1. The user signs in to the web or mobile application.
2. The user selects **Add device**.
3. The Control Plane creates a single-use pairing transaction with a two-minute expiry and a strict attempt limit.
4. The Gateway generates its device keypair locally.
5. The user enters the pairing code or scans the QR code from the workstation.
6. The Gateway sends the public key and pairing transaction proof to the Control Plane.
7. The KMS prepares a short-lived device certificate.
8. The Gateway displays a human-readable device fingerprint.
9. The trusted client displays the same fingerprint.
10. The user manually confirms that the fingerprints match.
11. The Control Plane marks the device trusted and establishes the encrypted tunnel.
12. The Audit Service records the pairing event.

The out-of-band fingerprint comparison is important because it prevents the relay from silently substituting a different public key during pairing. The user must be able to see and compare the value through two independent views.

11.2 Revocation

Any already-trusted user with sufficient permission should be able to revoke a device. Revocation must invalidate the device certificate, reject future tunnel authentication, terminate or quarantine active sessions, and create an audit event. A lost phone, compromised workstation, or former employee’ s device must not retain indefinite access.

---

12. Session and Task Execution

12.1 Start-session flow

When a user starts a task, the client sends the selected device, project, agent, prompt, trust profile, and requested capabilities to the Control Plane. The API validates the request and authorizes the user. The Control Plane creates the session record and writes an initial audit event before dispatching the task.

The broker delivers the command to the Gateway. The Gateway validates the message, confirms that the device and project are still available, asks the Sandbox Manager to create an isolated environment, loads the signed adapter, and starts the agent inside the sandbox. The adapter translates the user’ s prompt into the agent’ s native protocol.

12.2 Event flow

The agent emits output, tool events, file changes, and status updates. The adapter normalizes them. The Redaction Proxy scans and filters the outbound content. The Gateway assigns sequence numbers and sends encrypted event envelopes through the tunnel. The broker delivers them to the Control Plane, which updates the session projection, appends audit events where required, and streams the appropriate representation to connected clients.

12.3 Idempotency and ordering

Every command should contain a unique command ID, session ID, device ID, and sequence information. The Gateway must reject duplicate commands that have already been applied and must not execute the same commit, deployment, or destructive operation twice because of a network retry.

The Control Plane should maintain a durable command status such as `received`, `dispatched`, `acknowledged`, `applied`, `rejected`, or `expired`. For an action with irreversible consequences, the operation should require an explicit idempotency key and a final pre-execution check.

12.4 Cancellation

Cancellation should send a high-priority command to the Gateway. The Gateway terminates the adapter and sandbox process, records the reason, attempts cleanup, and reports the final state. If the Gateway is offline, the Control Plane records the cancellation request as pending and applies it when the device reconnects, subject to a configurable expiration.

---

13. Permission Engine and Approval Workflows

The Permission Engine balances automation and control. The system can auto-approve low-risk actions under a trust profile, but the immutable deny-override list establishes an approval floor that no profile may bypass.

13.1 Protected action categories

Category	Examples
Filesystem	Delete operations, writes outside the project, permission changes, ownership changes.

Shell	`sudo`, destructive commands, shell piping into an interpreter, operations outside the workspace.
Network	SSH, SCP, external POST requests, access to unapproved destinations.
Repository	Push to protected branches, force-push, history rewrite, tag changes.
Infrastructure	`terraform apply`, `kubectl apply`, production deployment, cloud-resource mutation.
Secrets	Reading credential files, exporting environment secrets, accessing private keys.

13.2 Deny-override list

The following operations always require explicit approval regardless of whether a session is marked autonomous:

- Force-push or history rewrite on a protected branch.
- Credential or secret export.
- Network activity that resembles secret exfiltration.
- Infrastructure apply against a production-tagged environment.

The list should be versioned, centrally managed, and protected from modification by ordinary workspace policies. An organization may add stricter rules, but it must not weaken the global floor.

13.3 Approval request lifecycle

When a protected operation is detected, the Gateway pauses the relevant execution point and sends a `PermissionRequest` containing the session, agent, action category, normalized command or description, target resource, risk explanation, requested capabilities, and expiration time.

The Control Plane evaluates policy and determines whether approval is required, who may approve, and whether escalation is available. The client receives the request and displays a clear decision screen. The user can approve, deny, or allow a narrowly scoped rule where permitted. A “remember this choice” option must never apply to deny-override actions.

Approval decisions use **first-writer-wins** semantics. The first valid decision resolves the request, and all connected devices immediately receive the resolved status. A later decision

is rejected as stale and cannot reverse the first decision.

13.4 Timeout and escalation

The safe default is automatic denial when the approval expires. Organizations may configure escalation to an authorized workspace administrator. Escalation must identify the required role and record the escalation path in the audit trail. Approval must never be silently granted because a user failed to respond.

---

14. Session State Machine

The session state machine makes the lifecycle explicit and allows the UI, notifications, and recovery logic to behave consistently.

State	Meaning
Offline	The Gateway or workstation is unavailable.
Idle	The device is online and can accept a new session.
Launching	The Gateway is creating the sandbox and starting the adapter.
Running	The agent is actively processing the task.
WaitingApproval	The agent is paused pending a policy decision.
Reconnecting	The tunnel has been interrupted and recovery is in progress.
Degraded	The connection gap exceeded the direct-resume threshold and reconciliation is required.
Completed	The task finished successfully.
Failed	The task ended because of an error, denied operation, crash, or unrecoverable condition.
Cancelled	The user or administrator explicitly stopped the session.

14.1 Key transitions

Plain Text

```
Idle -> Launching -> Running
Running -> WaitingApproval -> Running
WaitingApproval -> Failed          (denied or expired)
Running -> Reconnecting             (tunnel interruption)
Reconnecting -> Running             (successful short reconnect)
Reconnecting -> Degraded             (longer interruption)
Degraded -> Running                 (state reconciliation succeeds)
Running -> Completed                (successful finish)
Running -> Failed                   (agent or system error)
Any active state -> Cancelled       (authorized cancellation)
```

A reconnect within approximately sixty seconds may resume directly after sequence validation. A longer interruption must compare the Gateway’ s local checkpoint and event log with the last event acknowledged by the Control Plane. This prevents duplicated commands or missing output after a network gap.

---

15. Trust Profiles

Trust profiles control how aggressively low-risk actions are automated. They do not modify the deny-override list.

Profile	Intended behavior	Mandatory approval floor
Hands-On	Ask before commits, deployments, destructive actions, and other configured changes.	All deny-override actions.
Partial Trust	Auto-approve minor maintenance such as formatting or lint fixes; request approval for material changes.	Protected-branch force-push, secret export, and production infrastructure apply.
Autonomous	Permit safe maintenance tasks and selected auto-commits.	The immutable deny-override list.
AFK	Reduce automatic action while the user is away and raise notification priority for attention-required events.	Active-profile requirements plus time-sensitive or newly elevated actions.

Trust profiles should be scoped to a workspace, project, agent, or task rather than treated as a universal user setting. Each automatic decision must still be logged with the profile and policy version that produced it.

---

16. Attention Engine and Notifications

The Attention Engine prevents the user from being overwhelmed by routine agent output while ensuring that important events are not missed.

Tier	Examples	Default delivery
Tier 1: Critical	Approval request, successful completion, failure, device disconnection affecting a task.	Push notification and application badge.
Tier 2: Informational	Milestone, plan created, sub-agent started, major test phase completed.	In-app badge or activity feed.
Tier 3: Debug	Intermediate logs, token usage, verbose tool events.	Session console only.

Push messages should include only a short, redacted summary, session identifier, status, and a deep link. They should not include credentials, full source-code diffs, or sensitive command content. Users and organizations should be able to configure quiet hours, notification channels, batching, escalation, and severity thresholds.

---

17. Timeline, Audit, and Session Replay

The task timeline is both a user-facing history and a security record. It should present a chronological view of prompts, agent responses, tool invocations, approval requests, decisions, file changes, state transitions, notifications, errors, and access events.

17.1 Event structure

A normalized event should include:

Plain Text

```
id
workspaceId
sessionId
```

```
actorType
actorId
deviceId
eventType
payloadSummary
redactionStatus
sequenceNumber
timestamp
previousHash
currentHash
policyVersion
```

The payload should be minimized and redacted. A full command or diff may be stored locally or in encrypted storage according to policy, while the audit index stores a safe summary and cryptographic reference.

17.2 Tamper evidence

Events are linked using hashes. If a historical event is modified or removed, subsequent chain verification detects the break. This does not make the database impossible to compromise, so enterprise deployments should export audit data to an independent SIEM or write-once archive. The system should periodically publish chain checkpoints so later verification can compare independent copies.

17.3 Session replay

Authorized users should be able to replay a session as a timeline rather than re-execute it. Replay includes the event sequence, decisions, diffs, and state transitions. It must not accidentally expose secrets to users who did not have access at the time of execution.

---

18. Mobile-First and Web User Experience

The interface is designed for quick, safe decisions on a small screen while remaining useful in a desktop browser.

18.1 Main screens

Screen	Purpose
Dashboard	Shows paired devices, active sessions, attention items, and recent outcomes.
Device view	Displays connection state, operating system, Gateway version, and revocation controls.

Project view	Lists configured local projects and recent sessions.
Session list	Shows running, waiting, completed, failed, and degraded tasks.
Session detail	Combines chat, terminal output, progress, event timeline, and controls.
Approval screen	Explains the requested action, target, risk, policy reason, and decision options.
Diff review	Displays inline or side-by-side changes with file navigation and comments.
Integration screen	Manages repository, CI/CD, notification, and cloud connections.
Policy screen	Allows authorized users to manage trust profiles and workspace rules.
Audit screen	Searches, filters, verifies, and exports audit history.

18.2 Session detail

A session detail screen should show the current state prominently, followed by the most recent agent output, progress milestones, active capabilities, changed files, pending approvals, and an action bar. The user should be able to send a follow-up instruction, pause where supported, cancel, request a diff, or change notification behavior.

18.3 Diff review

The diff screen should support syntax highlighting, inline and side-by-side modes, file filtering, additions/deletions statistics, comments, and a clear distinction between proposed, approved, committed, and pushed changes. Commit and pull-request actions should be disabled until their policy checks complete.

18.4 Safe interaction design

Destructive actions must use explicit labels rather than ambiguous confirmation buttons. Approval dialogs should state what will happen, where it will happen, which identity requested it, and how long the decision remains valid. A mobile user should never have to infer whether “Allow” applies to one command or a broad future rule.

19. Integrations

19.1 Git providers

GitHub, GitLab, and Bitbucket integrations allow users to bind sessions to repositories, create branches, commit changes, push to permitted targets, and open pull requests. OAuth scopes should be minimal. Protected branches and force-push operations must remain subject to policy.

The platform should generate pull-request titles and descriptions from agent summaries, but the user must be able to edit them before submission. The PR should include a link to the session timeline and, where appropriate, an audit reference.

19.2 CI/CD

A CLI plugin or API integration should allow Jenkins, GitHub Actions, and similar systems to trigger sessions on a runner. CI-triggered sessions must use non-interactive policies with explicit timeouts and a restricted service identity. A deployment or production mutation must not bypass the same approval floor simply because the trigger came from automation.

19.3 IDE integrations

VS Code and JetBrains extensions can show active sessions, send follow-up instructions, open diffs, and link local files to remote review. The extension should communicate with the Gateway through the authenticated localhost API and should not receive broader permissions than the user’ s Control Plane session.

19.4 Slack and Teams

Slack and Teams can receive summaries, failure alerts, and approval notifications. For security, interactive approval links should return the user to the authenticated application or use a strongly authenticated workflow. Messaging platforms should not become an uncontrolled substitute for the main policy UI.

19.5 Cloud and infrastructure systems

Future integrations may support AWS, GCP, Kubernetes, Terraform, and MCP-based services. These integrations must treat production mutation as a high-risk capability. A staging dry run, plan output, or policy simulation should be available before a production apply is considered.

---

20. Organization, Workspace, and Role Model

The platform needs tenancy boundaries from the beginning because teams, administrators, and compliance users are part of the intended audience.

20.1 Core entities

Entity	Meaning
User	Human identity authenticated through the Auth Service.
Organization	Tenant boundary for team membership, policy, billing, and audit ownership.
Workspace	Logical development environment within an organization.
Project	Repository or local workspace associated with a Gateway.
Device	Paired workstation or approved client device.
Agent	Installed coding agent represented through an adapter.
Session	One execution instance of an agent task.
Approval	A policy-controlled decision request.
Integration	Authorized connection to an external service.
Audit event	Immutable record of an action or system transition.

20.2 Roles

- **Owner:** Controls organization ownership, security settings, membership, and irreversible administrative actions.
- **Admin:** Manages members, devices, policies, integrations, and escalated approvals according to organization rules.
- **Member:** Runs and reviews authorized sessions and approves actions within assigned scope.

Permissions should be resource-scoped. A team member may be allowed to inspect a project without being allowed to approve production deployment or revoke another administrator’ s device.

21. API and Data Model Outline

The first implementation should expose versioned APIs. Example resource groups include:

```
Plain Text

POST   /v1/pairing-transactions
POST   /v1/devices
GET    /v1/devices
POST   /v1/devices/{deviceId}/revoke
GET    /v1/projects
POST   /v1/sessions
GET    /v1/sessions/{sessionId}
POST   /v1/sessions/{sessionId}/messages
POST   /v1/sessions/{sessionId}/cancel
GET    /v1/sessions/{sessionId}/events
GET    /v1/sessions/{sessionId}/diffs
GET    /v1/approvals
POST   /v1/approvals/{approvalId}/decisions
GET    /v1/audit-events
POST   /v1/audit/verify
POST   /v1/integrations/{provider}/authorize
```

Illustrative durable tables include:

Table	Important fields
users	id, identityProvider, status, createdAt
organizations	id, name, policyVersion, createdAt
memberships	organizationId, userId, role, status
devices	id, organizationId, publicKey, certificateStatus, lastSeenAt
projects	id, deviceId, pathReference, repository, policyScope
agents	id, deviceId, adapterId, version, capabilities
sessions	id, projectId, agentId, state, trustProfile, startedAt, endedAt
commands	id, sessionId, idempotencyKey, status, sequenceNumber

approvals	id, sessionId, actionClass, decision, resolverId, expiresAt
diffs	id, sessionId, stepId, encryptedReference, hash
policies	id, organizationId, version, rules, denyOverrideVersion
auditEvents	id, sessionId, eventType, previousHash, currentHash, timestamp
integrations	id, organizationId, provider, encryptedCredentialReference

All tenant-owned records must include an organization or workspace boundary and be checked in every read and write path.

---

22. Security and Threat Model

The Gateway can control code execution, so it is a high-value target. The design adopts a zero-trust posture: no component is trusted merely because it is part of the platform or runs on the same machine.

22.1 Threats and mitigations

Threat	Mitigation
Stolen mobile device	MFA, short-lived sessions, device revocation, notification of suspicious access.
Compromised Gateway	Certificate revocation, tunnel termination, local least privilege, signed updates, session isolation.
Malicious relay or substituted key	Out-of-band fingerprint verification during pairing and end-to-end encryption.
Prompt injection through repository content	Treat code, files, logs, and external content as untrusted; enforce policy outside the agent.
Destructive agent command	Sandbox, policy classification, approval gate, deny-override list, resource limits.
Secret leakage in logs	Local secret injection, redaction proxy, sensitive-path rules, default-deny data

	handling.
Malicious adapter or binary	Signed packages, hash verification, version pinning, isolated execution.
Replay or duplicate command	Command IDs, sequence numbers, acknowledgements, idempotency keys.
Audit alteration	Hash chaining, append-only semantics, external SIEM export, chain verification.
Cross-tenant data access	Organization-scoped authorization and database isolation checks.
Pairing brute force	Two-minute expiry, single use, attempt limits, rate limiting, user alerts.
Network interruption	Local checkpoints, at-least-once delivery, reconciliation, explicit degraded state.

22.2 Defense-in-depth layers

The platform should enforce six independent layers:

1. **Network layer:** TLS, outbound-only Gateway connections, egress controls, and API rate limits.
2. **Identity layer:** OIDC, MFA, device keys, short-lived certificates, and revocation.
3. **Isolation layer:** Per-session sandboxing, filesystem scoping, resource quotas, and least privilege.
4. **Policy layer:** Central policy evaluation, approvals, trust profiles, and immutable deny overrides.
5. **Data layer:** End-to-end encryption, redaction, minimal retention, and secret isolation.
6. **Accountability layer:** Hash-chained audit logs, access records, export, and verification.

No single layer should be considered sufficient by itself.

---

23. Observability and Operations

23.1 Gateway metrics

The Gateway should expose local metrics for running sessions, agent CPU and memory use, sandbox resource consumption, event throughput, tunnel latency, reconnect frequency,

adapter failures, and redaction counts. Sensitive values must not be included in metrics labels.

23.2 Control Plane telemetry

The Control Plane should use structured logs and trace IDs. A session trace should cover the path from client request through API authorization, policy evaluation, broker dispatch, Gateway receipt, adapter execution, event delivery, and final audit append.

Key operational indicators include:

- Session start success rate.
- Median and tail latency from user command to Gateway receipt.
- Agent startup failure rate by adapter and version.
- Approval latency and timeout rate.
- Reconnect and degraded-session rate.
- Event delivery delay and duplicate rate.
- Sandbox creation and cleanup failures.
- Redaction detections.
- Device disconnect frequency.
- Audit-chain verification failures.

23.3 Alerting

The on-call system should alert on repeated Gateway failures, unexpected disconnects, elevated approval errors, broker backlog, certificate issuance anomalies, schema violations, audit-chain failures, sandbox escape indicators, and unusual cross-tenant access patterns.

---

24. Testing and Quality Strategy

Testing must cover both ordinary functionality and adversarial conditions.

Test area	Coverage
Unit testing	Adapters, policy rules, state transitions, redaction, serialization, authorization, and audit hashing.

Contract testing	Common adapter interface, broker messages, API schemas, and client event formats.
Integration testing	Gateway, Control Plane, broker, database, sandbox, and mock agent working together.
Approval testing	Protected action detection, approval timeout, escalation, first-writer-wins, and deny-override enforcement.
Resilience testing	Tunnel interruption, Gateway restart, duplicate delivery, delayed events, and state reconciliation.
Security testing	Prompt injection, command injection, sandbox escape, credential leakage, pairing attacks, and tenant isolation.
Supply-chain testing	Signature verification, rollback protection, corrupted packages, and adapter version pinning.
Performance testing	Concurrent sessions, event streaming, sandbox resource usage, broker throughput, and mobile latency.
Usability testing	One-handed mobile flows, approval clarity, terminal readability, diff review, and notification usefulness.
Release testing	Cross-platform installers, upgrades, certificate rotation, rollback, and uninstall cleanup.

Core modules should target at least 80% automated coverage, but coverage percentage must not replace security-focused scenario testing. The CI pipeline should run unit and integration tests on every change and schedule deeper security and resilience suites.

---

25. Deployment Model

25.1 Gateway distribution

The Gateway should be distributed as signed installers and binaries for Windows, macOS, and Linux. Package-manager distribution through Homebrew, apt, Chocolatey, or equivalent channels may improve adoption. The Gateway runs as a user-level background

service where possible, starts on boot according to user consent, listens only on localhost, and makes outbound connections.

Updates must be signature-verified and integrity-checked before installation. Users and administrators should be able to defer or disable automatic updates, subject to security policy. The updater should prevent downgrade attacks and provide a safe rollback path.

25.2 Cloud deployment

The Control Plane can run as containerized services on managed Kubernetes or a comparable orchestration environment. PostgreSQL should provide durable relational storage, Redis should provide ephemeral coordination, and a managed identity provider may provide OIDC and MFA.

A production deployment should include centralized secrets management, private service networking, encrypted storage, backup and restore procedures, disaster recovery objectives, multi-region planning, CDN delivery for web assets, and blue-green or canary releases.

25.3 Self-hosting

Enterprise customers may require the Control Plane to run in their own environment. The Gateway already supports on-premise execution because it runs on the customer’ s workstation. A self-hosted Control Plane guide should specify supported databases, broker configuration, KMS requirements, notification options, audit export, upgrades, and minimum security controls.

---

26. MVP Scope and Roadmap

26.1 MVP requirements

The MVP should include:

- Gateway for Windows, macOS, and Linux.
- Adapter support for Codex CLI, OpenCode, Claude CLI, and Antigravity CLI where their supported local interfaces permit integration.
- Device pairing with QR or one-time code.
- Out-of-band fingerprint confirmation.
- Short-lived device certificates and explicit revocation.
- Web Control Plane with device list and session list.
- Live session chat and terminal output.

- Session cancellation and reconnect handling.
- Mandatory per-session sandboxing.
- Default-deny network egress.
- Secret redaction proxy.
- Policy Engine with predefined protected actions.
- Immutable deny-override list.
- Approval workflow with first-writer-wins.
- Basic diff review.
- GitHub push and pull-request integration.
- MFA-based user authentication.
- Hash-chained audit events.
- Structured logs and basic metrics.
- Unit, integration, security-gate, and resilience tests.

26.2 Later releases

Later versions may add native iOS and Android applications, SSE fallback, Cline and Cursor adapters, IDE extensions, enhanced diff comments, group permissions, GitLab and Bitbucket, Slack and Teams, enterprise SSO and SCIM, FIDO2 approval confirmation, self-hosted Control Plane packaging, dashboards, scheduled tasks, webhook triggers, session analytics, multi-agent coordination, sub-agent spawning, and advanced attention rules.

The key sequencing principle is that security controls must not be postponed for convenience. Sandboxing, redaction, approval floors, device revocation, and audit integrity belong in the first secure release rather than being treated as enterprise-only polish.

---

27. Implementation Phases

Phase 1: Domain and security foundations

Define the organization, user, device, project, agent, session, approval, policy, and audit models. Establish authentication, tenant boundaries, device identity, key rotation, revocation, event schemas, and policy decision formats.

Phase 2: Gateway and sandbox foundation

Build the Gateway Core, local configuration, signed package verification, sandbox lifecycle, resource limits, project scoping, localhost authentication, checkpoint store, and tunnel client. Validate isolation before integrating complex agents.

Phase 3: First adapter and end-to-end session

Implement a mock echo agent and one production adapter. Build session creation, prompt delivery, normalized events, cancellation, completion, failure handling, and basic UI streaming. Use the mock agent to test all lifecycle paths deterministically.

Phase 4: Policy, approvals, and redaction

Implement action classification, approval requests, first-writer-wins, timeouts, escalation, deny overrides, secret redaction, and audit chaining. Add adversarial tests before enabling autonomous trust profiles.

Phase 5: Additional adapters and review experience

Add the remaining MVP adapters, diff capture, mobile-responsive review, commit controls, and GitHub integration. Track adapter-specific limitations using capability declarations.

Phase 6: Reliability, observability, and release hardening

Add reconnect reconciliation, metrics, tracing, alerts, cross-platform packaging, signed updates, performance testing, security review, backup and restore procedures, and release documentation.

Phase 7: Pilot and expansion

Run a controlled pilot with individual developers and small teams. Measure session success, approval clarity, security events, latency, adapter reliability, and retention. Expand integrations only after the core safety and reliability objectives are met.

---

28. Success Metrics and Acceptance Criteria

28.1 Product metrics

Area	Example measures
Adoption	Connected users, paired devices, active workspaces, active agents.

Engagement	Sessions per user per week, completed tasks, review actions, approvals.
Retention	Active users after one, three, and six months.
Performance	Command-to-agent latency, event delivery latency, reconnect recovery time.
Reliability	Session start success, adapter failure rate, broker delivery success, cleanup success.
Safety	Approval rate, denial rate, timeout rate, blocked actions, redaction events, security incidents.
Business	Individual conversion, team adoption, enterprise pilots, subscription or license revenue.

28.2 Technical acceptance criteria

The MVP should not be considered complete unless it can demonstrate that:

1. A paired Gateway can be revoked remotely and loses tunnel access.
2. A session cannot start an agent outside the required sandbox.
3. Default-deny network policy is enforced locally.
4. Secret-like values are removed before outbound streaming.
5. Deny-override actions require approval under every trust profile.
6. Concurrent approval decisions resolve deterministically.
7. A network interruption does not silently duplicate a command.
8. Audit-chain tampering is detectable.
9. One organization cannot retrieve another organization’ s sessions or audit data.
10. Gateway and adapter updates reject unsigned or modified packages.
11. The primary workflow works from a mobile-sized screen.
12. The system provides useful diagnostics when an adapter cannot support a requested capability.

---

29. Risks and Design Considerations

The largest risk is that the platform becomes a powerful remote-execution bridge whose security claims exceed what the Gateway can enforce. The project should therefore prioritize a small number of well-tested adapters and a strong sandbox boundary over a large but unreliable compatibility list.

Another risk is uneven agent support. Some agents expose stable APIs, while others require process management or may change their CLI behavior. The adapter contract must include version compatibility, capability discovery, health checks, and graceful degradation.

Mobile terminal interaction also presents a usability challenge. The product should not assume that reproducing a desktop terminal on a phone is always the best experience. Structured agent messages, clear milestones, compact logs, and safe action cards should complement the terminal rather than forcing users to read an uninterrupted stream.

Finally, encryption and redaction reduce the cloud’ s visibility but increase debugging complexity. The platform should maintain local diagnostic tools, safe correlation identifiers, explicit redaction reports, and user-controlled export mechanisms so failures can be investigated without exposing sensitive content.

---

30. Demonstration and Go-to-Market Concept

The strongest demonstration is an end-to-end scenario rather than a feature tour. A developer starts from a phone, selects a local workstation, asks the platform to repair a payment-validation bug, and launches two compatible agents in separate sandboxes. The agents inspect the repository and produce changes. The developer watches progress, receives an approval request for the commit, reviews the diff, approves it, and opens a pull request. CI runs normally, while the complete task history remains available in the session timeline.

The positioning should emphasize three differentiators:

- **One control layer across many coding agents.**
- **Local execution with remote supervision.**
- **Security controls that are architectural requirements rather than optional settings.**

Initial distribution can combine developer communities, technical content, live demonstrations, IDE marketplace integrations, and an individual free tier. Team and enterprise value should come from organization policy, audit, SSO, analytics, integrations, and self-hosting capabilities.

---

31. Final Summary

This project is a secure, vendor-neutral control plane for locally executed AI coding agents. Its two-part design combines a workstation-resident Gateway with a cloud Control Plane. The Gateway owns local execution, sandboxing, adapters, redaction, checkpoints, and the encrypted tunnel. The Control Plane owns authentication, device pairing, policy decisions, message delivery, notifications, integrations, durable task state, and audit history.

The enhanced architecture resolves the main weaknesses of the original concept by making several protections mandatory: per-session sandboxing, default-deny egress, secret redaction, out-of-band pairing verification, short-lived rotating certificates, device revocation, immutable approval floors, adapter supply-chain verification, first-writer-wins approval resolution, reconnect reconciliation, tenant isolation, and tamper-evident audit logs.

The central implementation strategy is to establish a stable normalized protocol and enforce security outside the agents themselves. New vendors can be integrated through adapters, while the Gateway and Control Plane maintain a consistent user experience. The product should launch with a carefully bounded MVP that proves secure pairing, isolated execution, real-time supervision, approval control, diff review, Git integration, and auditability before expanding into scheduling, multi-agent coordination, native mobile applications, enterprise identity, and broader integrations.

> **Implementation principle:** The platform may become more autonomous over time, but it must never become less accountable. Every remote action should be authorized, isolated, observable, recoverable, and reviewable.

Appendix A. Recommended Initial Technology Choices

The following choices are implementation recommendations rather than mandatory commitments. They provide a practical starting point for a production prototype.

Layer	Recommended starting approach
Gateway	Rust or Go for a portable, single-binary daemon with strong process and networking support.
Web Control Plane	TypeScript frontend with a typed API client and responsive mobile-first design.
Backend services	TypeScript, Go, or Rust with clear service interfaces and schema validation.
Durable database	PostgreSQL with organization-scoped access patterns and migrations.

Ephemeral state	Redis for presence, rate limits, short-lived locks, and broker coordination.
Live transport	WebSocket for interactive control, with an SSE or polling fallback later.
Message format	Versioned JSON or a compact typed binary envelope with sequence and idempotency fields.
Cryptography	Platform-reviewed standard libraries for X25519 key exchange and authenticated encryption; never custom cryptography.
Sandbox	Containers where available, otherwise operating-system sandboxing with documented platform limits.
Observability	OpenTelemetry-compatible traces, structured logs, and Prometheus-compatible metrics.
Deployment	Containers on managed Kubernetes or an equivalent managed runtime.
Mobile	Responsive web application first; native applications after the workflow is validated.

Technology selection must ultimately be validated against platform support, security review, distribution requirements, and the team’ s operational expertise.

Appendix B. Glossary

Term	Definition
AFK	Away from keyboard; a mode in which the user is not actively supervising from the workstation.
Adapter	A plugin that translates the platform’ s normalized protocol into a specific agent’ s CLI, HTTP, SDK, or process interface.
Agent Gateway	The local daemon that manages adapters, sandboxes, redaction, checkpoints, and the encrypted cloud tunnel.

Control Plane	The cloud services and user interfaces responsible for identity, orchestration, policy, notifications, integrations, and audit.
Deny override	A non-bypassable security rule requiring approval for a specified high-risk action.
Egress	Outbound network traffic from the agent sandbox.
Fingerprint	A human-verifiable representation of a device public key used during pairing.
Permission request	A structured request for authorization to perform a protected operation.
Redaction proxy	A local component that removes sensitive values from outbound agent data.
Session	One execution instance of an agent task against a selected project and device.
Trust profile	A configuration controlling which low-risk actions may be auto-approved.

Source Note

This document is a rewritten and expanded project specification based on the enhanced proposal supplied by the project owner. Competitive descriptions, supported-agent capabilities, and implementation decisions should be revalidated against current vendor documentation before production release, because agent APIs, CLIs, authentication methods, and licensing terms can change over time.